

Nefrock Internship memo

Executorch (XNNPACK)

executorchのbuild

- `./install_requirements.sh --pybind xnnpack`
- `./examples/models/llama2/install_requirements.sh`
 - l14: `--user`
- 注：ビルドキャッシュの削除
 - `./install_requirements.sh --clean`
 - `rm -rf $HOME/.buck *-out*`

モデルのエクスポート

- エクスポート

- `python -m examples.models.llama.export_llama -kv --use_sdpa_with_kv_cache -X --metadata '{"get_bos_id":128000, "get_eos_ids":[128009, 128001]}' -d bf16 --output_name="llama3_2_1b_1024.pt" --max_seq_length 1024 --checkpoint <checkpoint.pth> --params <params.json>`

- パラメータやトークナイザの転送

- `(adb shell mkdir -p /data/local/tmp/llama)`
- `adb push (path to file) /data/local/tmp/llama`

モデルのエクスポート (Quantization)

- SpinQuant

```
python -m examples.models.llama.export_llama --  
use_sdpa_with_kv_cache -X --xnnpack-extended-ops --preq_mode  
8da4w_output_8da8w --preq_group_size 32 -kv -d fp32 --  
preq_embedding_quantize 8,0 --use_spin_quant native --metadata  
'{"get_bos_id":128000, "get_eos_ids":[128009, 128001]}' --  
output_name "llama3_2_1b_i4_spin.pt" --max_seq_length 2048 --  
checkpoint <spinquant_checkpoint.pth> --params  
<spinquant_param.json>
```

- QNN

- 次のセクション

Androidアプリのbuild

- ANDROID_NDK, ANDROID_ABIを設定
- Build Android Java extension code

```
pushd extension/android
./gradlew clean
./gradlew build
popd
```
- Set up JNI library

```
pushd examples/demo-apps/android/LlamaDemo
./gradlew clean
./gradlew :app:setup
popd
```

Androidアプリのbuild (Quantization)

- <https://github.com/pytorch/executorch/tree/main/examples/models/llama#step-4-run-benchmark-on-android-phone>

例

- <https://github.com/pytorch/executorch/tree/main/examples/demo-apps/android/LlamaDemo>
- https://github.com/pytorch/executorch/blob/main/examples/demo-apps/android/LlamaDemo/docs/delegates/xnnpack_README.md

Executorch (QNN)

```
./install_requirements.sh
```

```
./backends/qualcomm/scripts/build.sh --release
```

```
cmake -DPYTHON_EXECUTABLE=python -DCMAKE_INSTALL_PREFIX=cmake-out -  
DEXECUTORCH_ENABLE_LOGGING=1 -DCMAKE_BUILD_TYPE=Release -  
DEXECUTORCH_BUILD_EXTENSION_MODULE=ON -DEXECUTORCH_BUILD_EXTENSION_DATA_LOADER=ON -  
DEXECUTORCH_BUILD_EXTENSION_TENSOR=ON -DEXECUTORCH_BUILD_QNN=ON -  
QNN_SDK_ROOT=${QNN_SDK_ROOT} -DEXECUTORCH_BUILD_KERNELS_QUANTIZED=ON -  
DEXECUTORCH_BUILD_KERNELS_OPTIMIZED=ON -DEXECUTORCH_BUILD_KERNELS_CUSTOM=ON -Bcmake-  
out .
```

```
cmake --build cmake-out -j16 --target install --config Release
```

```
sh examples/models/llama/install_requirements.sh
```

```
cmake -DPYTHON_EXECUTABLE=python -DCMAKE_INSTALL_PREFIX=cmake-out -  
DCMAKE_BUILD_TYPE=Release -DEXECUTORCH_BUILD_KERNELS_OPTIMIZED=ON -  
DEXECUTORCH_BUILD_KERNELS_QUANTIZED=ON -DEXECUTORCH_BUILD_KERNELS_CUSTOM=ON -  
DEXECUTORCH_BUILD_EXTENSION_TENSOR=ON -DEXECUTORCH_BUILD_QNN=ON -Bcmake-  
out/examples/models/llama examples/models/llama
```

```
cmake --build cmake-out/examples/models/llama -j16 --config Release
```

setup executorch

- `./install_requirements.sh`
- 注：ビルドキャッシュの削除
 - `./install_requirements.sh --clean`
 - `rm -rf $HOME/.buck *-out*`

setup QNN

- 環境変数の設定
- `cp schema/program.fbs exir/_serialize/program.fbs`
- `cp schema/scalar_type.fbs exir/_serialize/scalar_type.fbs`

QNN用executorchのビルド

- `./backends/qualcomm/scripts/build.sh --release`
- `cmake -DPYTHON_EXECUTABLE=python -
DCMAKE_INSTALL_PREFIX=cmake-out -
DEXECUTORCH_ENABLE_LOGGING=1 -
DCMAKE_BUILD_TYPE=Release -
DEXECUTORCH_BUILD_EXTENSION_MODULE=ON -
DEXECUTORCH_BUILD_EXTENSION_DATA_LOADER=ON -
DEXECUTORCH_BUILD_EXTENSION_TENSOR=ON -
DEXECUTORCH_BUILD_QNN=ON -
DQNN_SDK_ROOT=${QNN_SDK_ROOT} -
DEXECUTORCH_BUILD_KERNELS_QUANTIZED=ON -
DEXECUTORCH_BUILD_KERNELS_OPTIMIZED=ON -
DEXECUTORCH_BUILD_KERNELS_CUSTOM=ON -Bcmake-out .`
- `cmake --build cmake-out -j16 --target install --config Release`

setup Llama runner

- `sh examples/models/llama/install_requirements.sh`
- `cmake -DPYTHON_EXECUTABLE=python -
DCMAKE_INSTALL_PREFIX=cmake-out -
DCMAKE_BUILD_TYPE=Release -
DEXECUTORCH_BUILD_KERNELS_OPTIMIZED=ON -
DEXECUTORCH_BUILD_KERNELS_QUANTIZED=ON -
DEXECUTORCH_BUILD_KERNELS_CUSTOM=ON -
DEXECUTORCH_BUILD_EXTENSION_TENSOR=ON -
DEXECUTORCH_BUILD_QNN=ON -Bcmake-
out/examples/models/llama examples/models/llama`
- `cmake --build cmake-out/examples/models/llama -j16 --config
Release`

モデルのエクスポート (Quantization)

- PTQ
 - 4bit weight only
 - `python -m examples.models.llama.export_llama -kv --disable_dynamic_shape --qnn --pt2e_quantize qnn_16a4w -d fp32 --metadata '{"get_bos_id":128000, "get_eos_ids":[128009, 128001]}' --output_name="llama3_1_8b_ptq_qnn.pt" --max_seq_length 2048 --checkpoint <checkpoint.pth> -p <param.json>`

build App

- 環境変数の設定
- Build Android Java extension

```
pushd extension/android
./gradlew build
popd
```
- Setup JNI library
 - executorch/examples/demo-apps/android/LlamaDemo/app/build.gradle.kts: sh -> bash

```
pushd examples/demo-apps/android/LlamaDemo
./gradlew :app:setupQnn
popd
```
- Run App
 - ホスト側でadb server起動 (adb devicesなど)
 - docker側ではadb serverのホスト名を指定してadbコマンドを実行
 - adb -H docker.for.mac.localhost <command>

```
pushd examples/demo-apps/android/LlamaDemo
./gradlew :app:installDebug
popd
```


例

- https://github.com/pytorch/executorch/blob/main/examples/demo-apps/android/LlamaDemo/docs/delegates/qualcomm_README.md

docker上での環境構築

インストールしたソフトウェア

- apt
 - python3-dev (3.12以上)
 - python3-venv (仮想環境でないとexecutorchのビルド時にtorchのcmakelistを指定する必要があるそう)
 - openjdk-17-jdk
 - make
 - clang
- ダウンロード
 - android studio コマンドラインツール (sdkmanagerを含む)
 - QNN SDK v2.26

android studio コマンドラインツール

- sdkmanagerの使い方
 - `<android_sdk>/cmdline-tools/bin/sdkmanager --sdk_root=<android_sdk> (--install) "<package>"`
 - sdkmanagerコマンドでインストールしたソフトウェアは<android_sdk>下に配置される
 - デフォルトでsdkmanagerへのパスは設定されない
- sdkmanagerでのインストール（全て必要かは不明、バージョンはexecutorch参照）
 - `build-tools;34.0.0`（必要ないかも）
 - `build-tools;33.0.1`
 - `platform-tools`
 - `platforms;android-34`
 - `sources;android-34`
 - `ndk;27.1.12297006`（--installが必要）

llama.cpp

モデルのエクスポート

- Hugging Faceからレポジトリをクローン
- エクスポート
 - `python convert_hf_to_gguf.py <path to hf_repo> --outtype <type>`
- モデルの転送
 - `adb push <path to file>
/storage/emulated/0/Android/data/com.example.llama/files`